

Schedule

6-Week Intro to Databases & Data Engineering

Week	Agenda	Topics
1	Foundations	SQLite • Sets & relations • Logic basics • Keys
2	Basics in Modelling	Postgres • Subqueries • CTEs • JOINS • Normalisation
3	Dimensional Thinking	Postgres • Star schema • SCDs • Indexes • CASE
4	Document Thinking	MongoDB • Document models • Aggregation • CRUD
5	Graph Thinking	Postgres • Nodes/edges • Recursive CTEs
6	ACID + Spark	DataBricks • ACID • Transactions • Spark DataFrames

FCP and Scoring

Students will work in groups of four, forming a total of eleven groups (one group will have five members). The assignment will be released in the third lesson, with a submission deadline of 8 July. Deliverables must include code, documentation, and a presentation slide deck.

I'm open to your ideas — this project can genuinely become a strong experience to showcase on your CV. Think about what kind of hands-on work you'd like to be able to talk about in interviews, especially within this subject area. If you have suggestions or a direction you'd love to explore, send me an email at Yuanheng.Li.2@citystgeorges.ac.uk and please cc Yuanheng.Li@bayes.city.ac.uk. Thoughtful suggestions will earn you 1-2 bonus points.

WEEK 1: FOUNDATIONS + SQLITE

From Query Language to Query Engine

What Is SQL?

SQL (Structured Query Language) is a **declarative language** for manipulating and querying data in relational databases.

You describe **what** you want, not **how** to get it.

Declarative:

```
SELECT name, age FROM employees WHERE age > 30;
```

vs Imperative (pseudocode):

```
for each row in employees:  
    if row.age > 30:  
        output row.name, row.age
```

SQL abstracts away the "how."

Why SQL?

- We use SQL because real-world data is structured, relational, and needs guarantees like consistency and correctness.
- SQL is the practical language built on relational algebra — a mathematical system for manipulating sets of tuples.
- Whenever your data has entities and relationships, SQL is the right tool.

SQL as Functions

SQL clauses are **relational algebra operators**:

- **SELECT** = Projection function (pick columns)
- **WHERE** = Filter/Restriction function (pick rows)
- **JOIN** = Cartesian product + filter
- **GROUP BY** = Aggregation function
- **AS** = Naming/aliasing function

Composition: These functions compose to build queries.

```
SELECT name FROM employees WHERE dept = 'Eng';  
  |           |  
  |           +-- Filter(dept='Eng')  
+-- Project(name)
```

Relations: Tuples with Structure

A **relation** is a structured set of **tuples**.

Each tuple = fixed-length ordered collection of values.

Relation: EMPLOYEES

+-----+-----+-----+			
id	name	dept	
+-----+-----+-----+			
1	Alice	Eng	
2	Bob	HR	
+-----+-----+-----+			

<- Tuple: (1, Alice, Eng)

<- Tuple: (2, Bob, HR)

Schema defines tuple structure: Column names (attributes), Data types, and Constraints

Sets and Bags

Relational algebra - the theory behind SQL - operates on **sets** of tuples (unordered, no duplicates).

SQL implements these ideas but uses **bags (multisets)** by default, meaning duplicates are allowed unless you specify `DISTINCT`.

A database table is a **bag of rows**.

Example:

```
EMPLOYEES = {  
    (1, Alice, Eng),  
    (2, Bob, HR),  
    (3, Charlie, Sales)  
}
```


Primary Keys: Identity Function

A **primary key** is a function that maps each tuple to a unique identity.

Mathematically:

PK: Tuple $\rightarrow$ Unique Value (injective function)

In SQL:

```
CREATE TABLE employees (  
    emp_id INTEGER PRIMARY KEY,  ← PK function  
    name TEXT,  
    dept TEXT  
);
```

Property: emp_id(tuple) is unique for all tuples.

No two employees can have the same emp_id.

Projection: SELECT Function

SELECT extracts specific columns (attributes) from tuples.

Function:

```
Project(columns) : Relation -> Relation'
```

Examples:

```
SELECT name, dept FROM employees;  
-- Projects (name, dept) from (emp_id, name, dept)  
  
SELECT * FROM employees;  
-- Projects all columns (identity projection)
```

Result: New relation with fewer columns.

Restriction: WHERE Function

WHERE filters tuples based on a predicate (boolean condition).

Function:

```
Restrict(predicate) : Relation -> Relation'
```

Examples:

```
SELECT * FROM employees WHERE dept = 'Eng';  
-- Predicate: dept = 'Eng'  
-- Returns only matching tuples  
  
WHERE age > 30 AND salary > 100000;  
-- Composite predicate (logical AND)
```

Result: New relation with fewer rows.

Function Composition

SQL queries **compose** functions in a set order:

```
SELECT name, salary  
FROM employees  
WHERE dept = 'Eng' AND salary > 80000;
```

Composition:

```
employees -> Restrict(dept='Eng' AND salary>80000) -> Project(name, salary)
```

Order matters!

- Filter THEN project = smaller dataset projected
- Project THEN filter = pull in dept first, filter, and project again

SQL optimises automatically only when safe.

Introduction to SQLite

SQLite = SQL Engine embedded in a library.

Created by **D. Richard Hipp** (1999)

Key facts:

- Serverless (no daemon process)
- Single-file database (portable)
- Public domain (no license fees)
- Ubiquitous (iPhone, Android, Chrome, Firefox, Dropbox...)

Why "embedded"?

Your application links the SQLite library directly.

No separate database server to manage.

SQLite's Origin Story

Design goal:

Write SQL library for embedded systems without external dependencies or server requirements.

Result:

World's most widely deployed SQL database engine.

- Billions of devices
- Petabytes of data
- Zero configuration

SQLite in Real World

Where is SQLite?

- iMessages, WhatsApp messages (encrypted, stored locally)
- Firefox, Chrome browser databases
- iOS/Android apps (default)
- Dropbox, Google Drive (sync metadata)

Why SQLite?

- Lightweight (500 KB executable)
- Reliable (heavily tested, ACID)
- Portable (one file, copy anywhere)
- Zero admin overhead

SQLite Example: Web Browsers

```
-- Safari browser uses SQLite for History  
SELECT * FROM history;
```

All in one `.db` file in your user directory.

Query it yourself:

```
sqlite3 ~/Library/Safari/History.db
```


How SQL Becomes Executable

Pipeline:

```
SQL Text
|
[Lexer] <- Tokenization
|
[Parser] <- Syntax analysis
|
[AST] <- Abstract Syntax Tree
|
[Optimizer] <- Query optimization
|
[Executor] <- Code generation
|
Results
```

Lexer: Breaking into Tokens

Input:

```
SELECT name FROM employees WHERE age > 30;
```

Lexer output (tokens):

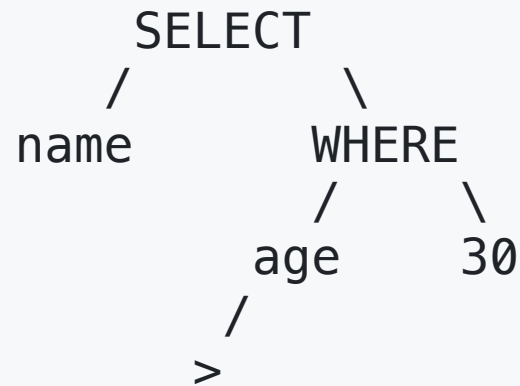
```
SELECT | name | FROM | employees | WHERE | age | > | 30 | ;
```

Each token has type (keyword, identifier, operator, literal).

Parser: Building the AST

Tokens become **Abstract Syntax Tree (AST)**.

AST represents structure hierarchically.



Matches SQL grammar rules.

Logical Plan: Converting AST -> Relational Algebra

Every SQL engine uses this logical order:

1. FROM
2. JOIN
3. WHERE
4. GROUP BY
5. HAVING
6. SELECT
7. ORDER BY
8. LIMIT

The database converts the AST into a logical plan:

```
employee
  |
Restrict(age > 30)
  |
Project(name)
```

Optimiser: Choosing Execution Plan

Multiple ways to execute same query:

Plan A: Filter THEN project (preferred)

```
Restrict(age > 30) -> Project(name)
```

Plan B: Project THEN filter (wasteful)

```
Project(name, age) -> Restrict(age > 30) -> Project(name)
```

Optimiser chooses Plan A (smaller intermediate results).

Executor: Generating Code

From Execution plan to machine code:

Optimiser produces **execution plan** with:

- Which indexes to use
- Join strategies
- Memory allocation
- Iteration order

Executor runs this plan on actual data.

CREATE TABLE: Defining Relations

```
CREATE TABLE employees (  
    emp_id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL,  
    dept TEXT,  
    salary REAL  
);
```

```
CREATE TABLE department (  
    dept_id INTEGER PRIMARY KEY,  
    dept_name TEXT NOT NULL  
);
```


INSERT: Building the Set

```
INSERT INTO employees (emp_id, name, dept, salary)
VALUES
  (1, 'Alice', 'Engineering', 95000),
  (2, 'Bob', 'Sales', 80000),
  (3, 'Carol', 'Engineering', 100000);
```

```
INSERT INTO department (dept_id, dept_name)
VALUES
  (10, 'Engineering'),
  (20, 'Sales');
```

```
ALTER TABLE employees ADD COLUMN dept_id INTEGER;  
UPDATE employees  
SET dept_id = (  
    SELECT dept_id  
    FROM department  
    WHERE department.dept_name = employees.dept  
);
```

SELECT: Query the Set

```
SELECT name, salary FROM employees  
WHERE dept = 'Engineering';
```

Execution:

1. Restrict by dept='Engineering' -> 2 tuples
2. Project (name, salary) -> 2x2 relation

Result:

Alice	95000
Carol	100000

JOIN: Cartesian Product + Filter

Cartesian Product = All combinations.

Example tables:

```
Depts: {(10, Engineering), (20, Sales)}  
Locs: {(NYC), (London)}
```

Cartesian Product:

```
(10, Engineering, NYC)  
(10, Engineering, London)  
(20, Sales, NYC)  
(20, Sales, London)
```

JOIN = Cartesian + WHERE filter

```
INSERT INTO employees VALUES (4, 'David', 'HR', 100000, 3);  
INSERT INTO department VALUES (4, 'Finance');
```

INNER JOIN Example

```
SELECT e.name, d.dept_name  
FROM employees e  
INNER JOIN department d ON e.dept_id = d.dept_id;
```

Steps:

1. Cartesian: employees $\times$ department (N $\times$ M tuples)
2. Filter: WHERE e.dept_id = d.id
3. Project: e.name, d.dept_name

Result: Only matching pairs.

LEFT JOIN: All Left + Matching Right

```
SELECT e.name, d.dept_name  
FROM employees e  
LEFT JOIN department d ON e.dept_id = d.dept_id;
```

Difference:

- INNER: Only matching rows
- LEFT: All employees, even without department

Unmatched columns get NULL.

FULL OUTER JOIN in SQLite

Older version do not support FULL OUTER JOIN but recent versions do.

```
SELECT e.name, d.dept_name  
FROM employees e  
FULL OUTER JOIN department d ON e.dept_id = d.dept_id;
```


FULL OUTER JOIN

```
Alice (dept 10)
Bob (dept 20)
Carol (dept 10)
David (dept 3)  <- No dept exists!
```

```
1: Engineering
2: Sales
4: Finance  <- No employee!
```

FULL OUTER JOIN result:

Alice	Engineering
Bob	Sales
Carol	Engineering
David	NULL
NULL	Finance

Old approach: Implementing FULL OUTER JOIN

Step 1: INNER JOIN = CROSS JOIN + FILTER

```
SELECT e.name, d.dept_name  
FROM employees e  
CROSS JOIN department d  
WHERE e.dept_id = d.dept_id;
```

Step 2: LEFT EXCLUSIVE (unmatched employees)

```
SELECT e.name, d.dept_name
FROM employees e
LEFT JOIN dept d ON e.dept_id = d.dept_id
WHERE d.name IS NULL;
```

```
-- Query 1 UNION ALL
SELECT e.name, NULL
FROM employees e
WHERE NOT EXISTS (
    SELECT 1
    FROM department d
    WHERE d.dept_id = e.dept_id
);
```

FULL OUTER JOIN (Part 3)

Older version of SQLite doesn't support `RIGHT JOIN` syntax directly.

Step 3: RIGHT exclusive (unmatched depts)

```
SELECT e.name, d.dept_name
FROM employees e
RIGHT JOIN department d ON e.dept_id = d.dept_id
WHERE e.name IS NULL;
```

```
-- Query 2 UNION ALL
SELECT NULL AS name, d.dept_name
FROM department d
WHERE d.dept_id NOT IN (
    SELECT e.dept_id FROM employees e
);
```

FULL OUTER JOIN (Part 4)

Step 4: UNION ALL

```
SELECT e.name, d.dept_name
FROM employees e
CROSS JOIN department d
WHERE e.dept_id = d.dept_id
UNION ALL
SELECT e.name, NULL
FROM employees e
WHERE NOT EXISTS (
    SELECT 1
    FROM department d
    WHERE d.dept_id = e.dept_id
)
```

```
UNION ALL
SELECT NULL, d.dept_name
FROM department d
WHERE NOT EXISTS (
    SELECT 1
    FROM employees e
    WHERE e.dept_id = d.dept_id
);
```

Complete result:

Alice	Engineering
Bob	Sales
Carol	Engineering
David	NULL
NULL	Finance

Why FULL OUTER JOIN Matters

Use case: Data reconciliation

Accounts in System A: {Alice, Bob, Carol}
Accounts in System B: {Alice, Bob, David}

FULL OUTER JOIN finds:

- Both systems: Alice, Bob (matching)
- System A only: Carol (needs migration)
- System B only: David (duplicate removal)

Query Execution: Real World

Your query:

```
SELECT name FROM employees WHERE dept = 'Engineering' ORDER BY name;
```

What SQLite actually does:

1. Tokenize (SQL -> tokens)
2. Parse (tokens -> AST)
3. Validate (schema check)
4. Optimize (best execution plan)
5. Execute (iterate through data)
6. Sort (ORDER BY)
7. Return results

Performance: Using Indexes

Without index:

```
SELECT * FROM employees WHERE dept_id = 1;  
-- Must read all 1M rows to find matches
```

With index:

```
CREATE INDEX idx_dept ON employees(dept_id);  
SELECT * FROM employees WHERE dept_id = 1;  
-- Jumps directly to matching rows (10x faster)
```

AST optimizer recognizes indexable conditions.

Why This Matters

Understanding the **pipeline**:

- Lexer/Parser/AST = Foundation
- Functions (SELECT, WHERE, JOIN) = Composable logic
- Optimization = Why queries perform

You become a better SQL writer when you understand what happens under the hood.

Exercise 1: Create and INSERT

Create tables:

```
CREATE TABLE customers (  
    cust_id INT PRIMARY KEY,  
    name TEXT  
);  
  
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    cust_id INT,  
    amount REAL  
);
```

Insert data:

```
INSERT INTO customers VALUES  
  (1, 'Alice'),  
  (2, 'Bob'),  
  (3, 'Charlie');    -- unmatched customer
```

```
INSERT INTO orders VALUES  
  (101, 1, 100),  
  (102, 1, 200),  
  (103, 2, 150),  
  (104, 99, 500);    -- unmatched order
```

Exercise 2: Basic Queries

Write these queries:

1. All customers
2. Orders over 150
3. Customer names with their orders (JOIN)
4. Customers without orders (LEFT JOIN + NULL check)

Exercise 3: FULL OUTER JOIN

Implement FULL OUTER JOIN:

```
-- All customers and their orders (including those with no orders)  
-- AND all orders (including those from non-existent customers)
```

Hint: Use LEFT JOIN + UNION + RIGHT exclusive.

Or INNER JOIN + LEFT exclusive + RIGHT exclusive.

Exercise 4: Teach yourself

For this query:

```
SELECT c.name, COUNT(o.order_id)
FROM customers c
LEFT JOIN orders o ON c.cust_id = o.cust_id
GROUP BY c.cust_id
HAVING COUNT(o.order_id) > 0;
```

Put this into AI tools you like and let it explain to you in natural language

Exercise 5: Performance Analysis

Query:

```
SELECT * FROM orders WHERE cust_id = 1;
```

Questions:

1. Think how many rows scanned (with/without index)
2. How would you optimize?
3. When would ORDER BY change the execution plan?

How to add an index:

```
CREATE INDEX idx_orders_cust_id ON orders(cust_id);
```

Case A — ORDER BY matches the index

```
SELECT * FROM orders  
WHERE cust_id = 1  
ORDER BY amount;
```

If you add:

```
CREATE INDEX idx_orders_cust_amount ON orders(cust_id, amount);
```

SQLite can:

- Use the index to find matching rows
- Return them already sorted
- Avoid a sort step

Case B — ORDER BY on a non-indexed column

SQLite must:

- Fetch rows
- Then sort them manually
- So it needs more CPU + memory

Case C — ORDER BY forces a different join strategy

With index only on `cust_id`

```
SELECT c.name, o.amount
FROM customers c
JOIN orders o ON c.cust_id = o.cust_id
WHERE c.cust_id < 3;
-- ORDER BY o.amount
```

SQLite may switch to:

- Scan orders first
- Sort by amount
- Then join to customers

Key Takeaways

SQL is:

- **Declarative** (what, not how)
- **Functional** (operations compose)
- **Mathematical** (sets, relations, functions)

SQLite is:

- **Embedded** (no server)
- **Ubiquitous** (billions of devices)
- **Practical** (query your own device)

Understanding the pipeline (Lexer → AST → Optimizer → Executor)
makes you a better developer.

Resources

SQLite:

- Official: <https://sqlite.org>
- Tutorial: <https://sqlitetutorial.net>
- Browser in your machine: Query history databases!

Next week: Multi-table design and normalization.